Teoriuppgifter labb 3

Joel Celinder

Let n be the number of vertices and m the number of edges.

1. Adjacency matrix vs. adjacency lists

Adjacency matrix

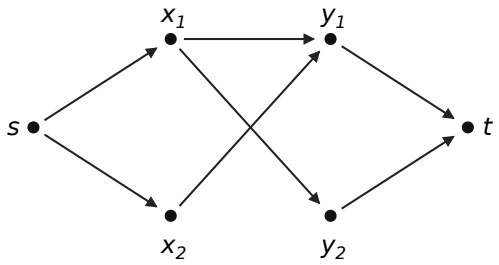
Finding the neighbours of a vertex requires scanning its entire row, which takes $\Theta(n)$ per vertex. A BFS dequeues each vertex at most once, so one BFS takes $O(n^2)$. Edmonds-Karp performs $O(n \cdot m)$ augmentations, each requiring one BFS, so the total time complexity is $O(n \cdot m) \cdot O(n^2) = \mathbf{O(n^3 \cdot m)}$.

Adjacency lists

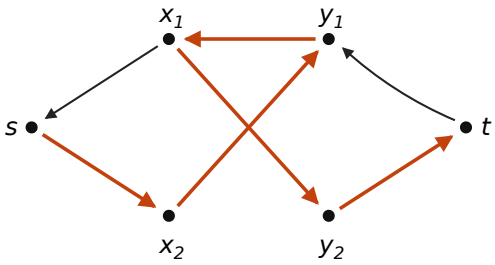
Finding the neighbours of a vertex u requires walking through its list, which takes $\Theta(\deg(u))$. One BFS takes $O(n + \Sigma \deg(u)) = O(n + m)$, which is $O(m)$ since we may assume every vertex is reachable from s (so $m \geq n - 1$). With $O(n \cdot m)$ augmentations the total is $O(n \cdot m) \cdot O(m) = \mathbf{O(n \cdot m^2)}$.

When $m = O(n)$, the adjacency matrix gives $O(n^4)$ while the adjacency lists give $O(n^3)$, so the lists are better when the graph is sparse. The $O(n \cdot m^2)$ bound for the lists assumes that each edge has a pointer to its reverse edge, so that $f[v,u] := -f[u,v]$ can be done in $O(1)$. Without it, finding the reverse edge would require searching v's list.

2. Length of the augmenting paths



H: every s→t path has length 3



Residual graph after matching x_1–y_1: the only path is $s \to x_2 \to y_1 \to x_1 \to y_2 \to t$, length 5

After a few augmentations some of the edges s→x, x→y and y→t are full, and the residual graph has reverse edges y→x for the matched pairs. So if a free left vertex only has edges to right vertices that are already matched, BFS can't go straight to t. It has to go to a matched y, back along the reverse edge to y's partner, and keep going until it reaches a free right vertex. The path zigzags between the two sides.

Let X and Y be the left and right sides of G. The path can use at most $\min(|X|,|Y|)$ vertices from each side, so the longest possible path has length $2 \cdot \min(|X|,|Y|) + 1 \leq n + 1$, where n is the number of vertices in G.

3. Integer flows

a) Proof by induction on the number of augmentations. Initially the flow is 0 in every edge, which is an integer. Assume all flows are integers before an augmentation. Then all residual capacities, $c(u,v) - f(u,v)$ and $f(u,v)$, are integers too. The augmenting path only uses edges with positive residual capacity, so the bottleneck is the minimum of some positive integers, i.e. an integer ≥ 1. The flow in each edge on the path changes by exactly the bottleneck, so all flows are still integers after the augmentation. Since the flow increases by at least 1 each time and is bounded by the sum of the capacities out of s, the algorithm terminates, and the final flow is an integer flow.

b) The flows are still integers, but the flow into a left vertex x can now be 2. Since x→y and y→t still have capacity 1, x can send one unit each to two different right vertices, while every right vertex still gets at most one unit. So the solution is no longer a matching: each left vertex can be paired with up to two right vertices, and each right vertex with at most one left vertex.